

1. Why Strings Matter in Data Engineering

Strings are used everywhere in Data Engineering.

Typical examples:

- File names
- Folder paths
- Table names
- Column names
- API URLs
- JSON keys
- Pipeline names
- Status values
- Error messages
- Dates received as text
- Log messages

A Data Engineer should be able to:

- Inspect strings
 - Clean strings
 - Extract useful information
 - Validate patterns
 - Standardise source values
-

2. What Is a String?

A string is a sequence of characters.

```
pipeline_name = "customer_pipeline"
```

Check the type:

```
print(type(pipeline_name))
```

Output:

```
<class 'str'>
```

Strings can use single or double quotes:

```
source_name = "customer_api"  
target_table = 'customers'
```

3. Multiline Strings

Triple quotes can be used for long text.

```
pipeline_description = """  
This pipeline reads customer data,  
validates the schema,  
and loads it into the target.  
"""
```

Useful for:

- SQL queries
 - Long log messages
 - Documentation
 - Configuration text
 - API request bodies
-

4. String Length

Use `len()` to count characters.

```
file_name = "customers.csv"  
  
print(len(file_name))
```

Output:

Spaces and special characters are also counted.

Example:

```
error_message = ""  
  
print(len(error_message))
```

Output:

0

This helps identify empty values.

5. String Indexing

Python indexing starts from 0.

```
file_type = "csv"
```

Positions:

```
c → 0  
s → 1  
v → 2
```

Example:

```
print(file_type[0])
```

Output:

c

Negative Indexing

Negative indexes start from the end.

```
file_name = "customers.csv"
```

```
print(file_name[-1])
```

Output:

```
v
```

Important:

Accessing a position that does not exist causes an `IndexError`.

6. String Slicing

Slicing extracts part of a string.

Syntax:

```
value[start:end]
```

The start index is included, but the end index is excluded.

Example:

```
file_name = "customers.csv"
```

```
entity_name = file_name[:9]
```

```
print(entity_name)
```

Output:

```
customers
```

Extract extension:

```
file_extension = file_name[-4:]
```

Output:

```
.csv
```

Important

Hardcoded slicing works only when the string pattern is fixed.

For filenames with varying entity-name lengths, methods such as `split()` are usually safer.

7. Strings Are Immutable

Strings cannot be changed character by character.

Incorrect:

```
file_name = "customers.csv"
```

```
file_name[0] = "C"
```

This fails.

Instead, Python creates a new string.

```
file_name = "C" + file_name[1:]
```

Output:

```
Customers.csv
```

Most string methods also return a new string.

8. Case Standardisation

Source systems may send:

SUCCESS
Success
success
SuCcEsS

Python considers them different.

```
print("SUCCESS" == "success")
```

Output:

False

A safer comparison:

```
source_status = "Success"  
  
is_success = source_status.lower() == "success"
```

Output:

True

lower()

Converts text to lowercase.

```
file_name = "CUSTOMERS.CSV"  
  
file_name = file_name.lower()
```

Result:

customers.csv

upper()

Converts text to uppercase.

```
pipeline_status = "success"
```

```
pipeline_status = pipeline_status.upper()
```

Result:

SUCCESS

9. Removing Extra Spaces

Source data often contains leading or trailing spaces.

```
customer_name = " Anurag "
```

Clean using:

```
customer_name = customer_name.strip()
```

Result:

Anurag

Other methods:

lstrip()

rstrip()

- `lstrip()` removes left-side spaces.
- `rstrip()` removes right-side spaces.
- `strip()` removes both.

For most cleaning scenarios, `strip()` is the common choice.

10. `replace()` for Standardisation

Suppose a source column is:

```
column_name = "customer name"
```

The target expects:

```
customer_name
```

Use:

```
column_name = column_name.replace(" ", "_")
```

Another example:

```
column_name = "order-amount"
```

```
column_name = column_name.replace("-", "_")
```

Result:

```
order_amount
```

11. Method Chaining

Multiple cleaning operations can be combined.

```
column_name = " Customer-Full Name "
```

```
clean_column_name = (  
    column_name  
    .strip()
```



```
.lower()  
.replace("-", "_")  
.replace(" ", "_")  
)
```

Result:

customer_full_name

This is called **method chaining**.

12. Handling Repeated Spaces

Suppose:

```
column_name = "Customer  Full  Name"
```

Using:

```
column_name.replace(" ", "_")
```

can produce multiple underscores.

A better approach:

```
clean_column_name = "_".join(  
    column_name.strip().lower().split()  
)
```

Result:

customer_full_name

Why?

First:

```
column_name.strip().lower().split()
```

creates separate words:

```
["customer", "full", "name"]
```

Then:

```
"_".join(...)
```

combines them using underscores.

13. `startswith()` for File Identification

Example:

```
file_name = "customers_2026_07_31.csv"
```

```
is_customer_file = file_name.startswith("customers")
```

Output:

```
True
```

Useful when different file types need different pipeline logic.

Example:

```
if file_name.startswith("orders"):
    print("Order file received")
```

14. `endswith()` for File Validation

Use `endswith()` when checking file extensions.

```
file_name = "customers_2026_07_31.csv"
```

```
is_csv = file_name.endswith(".csv")
```

Output:

True

This is safer than:

"`.csv`" in `file_name`

Because:

```
file_name = "customers.csv.backup"
```

contains `.csv`, but does not actually end with `.csv`.

Multiple Supported Extensions

```
file_name = "customers.json"
```

```
is_supported = file_name.endswith(  
    ".csv", ".json", ".parquet"  
)
```

Output:

True

15. `find()` Method

`find()` returns the position of the first matching value.

```
file_name = "customers_2026_07_31.csv"
```

```
position = file_name.find("_")
```

Output:

9

If the value does not exist:

```
file_name.find("-")
```

returns:

-1

Example: extracting entity name dynamically:

```
first_underscore = file_name.find("_")
```

```
entity_name = file_name[first_underscore]
```

Result:

customers

This is more flexible than hardcoding a fixed character position.

16. `split()` Method

`split()` divides one string into multiple parts.

```
file_name = "customers_2026_07_31.csv"
```

```
file_parts = file_name.split("_")
```

Result:

```
[  
    "customers",  
    "2026",  
    "07",  
    "31.csv"  
]
```

You can then extract:

```
entity_name = file_parts[0]  
year = file_parts[1]  
month = file_parts[2]  
day_with_extension = file_parts[3]
```

17. `rsplit()` for File Extension

Suppose:

```
file_name = "customer.data.2026.csv"
```

Using:

```
file_name.split(".")
```

creates multiple parts.

If only the final extension is needed:

```
name_part, extension = file_name.rsplit(".", 1)
```

Result:

```
name_part = customer.data.2026  
extension = csv
```

This is useful when filenames contain multiple dots.

18. `join()` Method

`join()` combines multiple string values.

Example:

```
path_parts = [  
    "landing",  
    "customers",  
    "2026",  
    "07",  
    "31"  
]
```

```
folder_path = "/".join(path_parts)
```

Output:

```
landing/customers/2026/07/31
```

Useful for:

- Folder paths
- Storage paths
- Partition paths
- Column lists
- Log messages

19. Useful Validation Methods

`count()`

Counts occurrences.

```
file_name = "customers_2026_07_31.csv"
```

```
print(file_name.count("_"))
```

Output:

3

Can help validate filename patterns.

isdigit()

Checks whether all characters are digits.

```
year = "2026"
```

```
print(year.isdigit())
```

Output:

True

But:

```
"20A6".isdigit()
```

returns:

False

Important:

isdigit() checks characters only.

It does **not** validate whether a month or date is logically correct.

isalpha()

Checks whether all characters are letters.

```
"customers".isalpha()
```

returns:

True

But:

```
"customer_data".isalpha()
```

returns:

False

because `_` is not alphabetic.

isalnum()

Checks whether all characters are letters or numbers.

```
"BATCH101".isalnum()
```

returns:

True

But:

```
"BATCH-101".isalnum()
```

returns:

False

20. Real Scenario: Filename Parsing

Suppose the source file is:

```
file_name = " Customers_2026_07_31.CSV "
```

Expected pattern:

```
entity_year_month_day.extension
```

Step 1: Clean the Filename

```
file_name = file_name.strip().lower()
```

Result:

```
customers_2026_07_31.csv
```

Step 2: Separate Extension

```
name_without_extension, extension = file_name.rsplit(".", 1)
```

Result:

```
name_without_extension = customers_2026_07_31  
extension = csv
```

Step 3: Split Components

```
file_parts = name_without_extension.split("_")
```

Result:

```
["customers", "2026", "07", "31"]
```

Step 4: Extract Values

```
entity_name = file_parts[0]  
year = file_parts[1]  
month = file_parts[2]  
day = file_parts[3]
```

Step 5: Validate

```
valid_extension = file_name.endswith(".csv")  
valid_entity = entity_name == "customers"  
valid_year = year.isdigit()  
valid_month = month.isdigit()  
valid_day = day.isdigit()  
valid_part_count = len(file_parts) == 4
```

Final validation:

```
file_name_valid = (  
    valid_extension  
    and valid_part_count  
    and valid_entity  
    and valid_year  
    and valid_month  
    and valid_day  
)
```

21. Important Limitation

This filename:

customers_2026_15_45.csv

may still pass:

```
month.isdigit()  
day.isdigit()
```

because both contain digits.

But month 15 and day 45 are not valid calendar values.

So remember:

String validation checks the pattern, not necessarily the business validity of the date.

22. Real Scenario: Column Standardisation

Raw source column:

```
raw_column_name = " Customer Full-Name "
```

Target standard:

```
customer_full_name
```

Clean it:

```
clean_column_name = (  
    raw_column_name  
    .strip()  
    .lower()  
    .replace("-", " ")  
)
```

```
clean_column_name = "_".join(  
    clean_column_name.split()  
)
```

Result:

```
customer_full_name
```

This is a common schema-cleaning activity before loading source data into a target table.

23. Creating Dynamic Paths

Suppose:

```
entity_name = "customers"  
year = "2026"  
month = "07"  
day = "31"
```

Create a partition-style path:

```
folder_path = (  
    f"landing/{entity_name}/"  
    f"year={year}/month={month}/day={day}"  
)
```

Output:

```
landing/customers/year=2026/month=07/day=31
```

24. Common Mistakes

Mistake 1: Forgetting Indexing Starts at 0

```
value = "csv"
```

```
value[0]
```

gives the first character.

Mistake 2: Invalid Index

```
value = "csv"
```

```
value[5]
```

causes `IndexError`.

Mistake 3: Assuming Methods Modify the Original String

```
file_name = "CUSTOMERS.CSV"
```

```
file_name.lower()
```

```
print(file_name)
```

Still prints:

```
CUSTOMERS.CSV
```

Correct:

```
file_name = file_name.lower()
```

Mistake 4: Using `in` Instead of `endswith()`

Risky:

```
".csv" in file_name
```

Better:

```
file_name.endswith(".csv")
```

Mistake 5: Hardcoding Positions

Avoid:

```
entity_name = file_name[:9]
```

Prefer delimiter-based extraction:

```
entity_name = file_name.split("_")[0]
```

when the filename follows a known pattern.

25. Quick Revision

- Strings are widely used in filenames, paths, columns and logs.
 - `len()` gives string length.
 - Indexing starts from 0.
 - Negative indexing starts from the end.
 - Slicing extracts part of a string.
 - Strings are immutable.
 - `lower()` and `upper()` standardise case.
 - `strip()` removes surrounding spaces.
 - `replace()` replaces characters or text.
 - `startswith()` helps identify file types/entities.
 - `endswith()` is useful for extension validation.
 - `split()` breaks a string into parts.
 - `rsplit()` is useful when splitting from the right.
 - `join()` combines strings.
 - `isdigit()` checks numeric characters.
 - Avoid hardcoded positions when delimiter-based parsing is possible.
-

26. Student Practice Assignment 1

Use:

```
file_name = "Orders_2026_08_15.JSON "
```

Clean and extract:

- Entity name
- Year
- Month

- Day
- Extension

Validate:

- Entity should be `orders`
- Extension should be `.json`
- Filename should contain four underscore-separated components
- Year, month and day should contain digits

Expected cleaned filename:

`orders_2026_08_15.json`

Expected values:

Entity: `orders`

Year: `2026`

Month: `08`

Day: `15`

Extension: `json`

27. Student Practice Assignment 2

Standardise:

`column_1 = " Customer ID "`

`column_2 = "Customer Full Name"`

`column_3 = "ORDER-AMOUNT"`

`column_4 = " Customer Email Address "`

`column_5 = "Product-Category Name"`

Expected:

`customer_id`

`customer_full_name`

`order_amount`

`customer_email_address`

`product_category_name`

Try using:

- `strip()`
 - `lower()`
 - `replace()`
 - `split()`
 - `join()`
-

28. Interview-Relevant Questions

1. What is a string in Python?

A string is an immutable sequence of characters used to represent values such as filenames, paths, column names, statuses and log messages.

2. What does string immutability mean?

It means an existing string cannot be changed character by character. String operations return a new string.

3. What is the difference between indexing and slicing?

Indexing retrieves one character. Slicing retrieves a range of characters.

4. Why is `endswith()` better than `in` for extension validation?

`in` checks whether text appears anywhere in the filename, while `endswith()` confirms the actual ending.

5. What does `strip()` do?

It removes spaces or specified characters from the beginning and end of a string.

6. Difference between `split()` and `join()`?

`split()` breaks one string into multiple parts.

`join()` combines multiple strings into one string.

7. What does `find()` return when a value is absent?

-1

8. Does `lower()` modify the original string?

No. Strings are immutable. The returned value must be assigned if you want to keep the change.

9. How would you standardise a column name like " Customer Full Name " ?

```
clean_name = "_".join(
    column_name.strip().lower().split()
)
```

10. Why are string operations important for Data Engineers?

Because Data Engineers regularly parse filenames, clean columns, validate extensions, create paths, standardise status values and process raw text fields.

29. Mini Interview Challenge

Predict the result:

```
value = " Data Engineering "
```

```
print(value.startswith("Data"))
print(value.strip().startswith("Data"))
```

Expected:

False
True

Why?

The original string starts with a space.

After `strip()`, the extra space is removed.